

Manual de Despliegue

World Cup Predictor — Spring Boot + Vanilla JS

Guia completa para poner el proyecto en internet

1. Arquitectura del proyecto

El proyecto tiene **tres componentes** que necesitan desplegarse de forma independiente o coordinada. Cada uno tiene requisitos distintos de hosting:

COMPONENTE	TECNOLOGIA	PUERTO LOCAL	QUE NECESITA EN PRODUCCION
Frontend	HTML / CSS / JS puro	8000	Cualquier servidor de archivos estaticos (Netlify, GitHub Pages)
Backend	Java 17 + Spring Boot 3.2	8080	Servidor con JVM (Railway, Render, Oracle Cloud)
Base datos	H2 embebida (archivo local)	—	Migrar a PostgreSQL en produccion (Railway, Supabase)

2. Prerequisitos para ejecutar en local

HERRAMIENTA	VERSION MINIMA	PARA QUE SE USA
Java JDK	17 o superior	Compilar y ejecutar el backend Spring Boot
Maven	3.8+	Gestionar dependencias y empaquetar el .jar del backend
Node.js	18+	Ejecutar el servidor de desarrollo del frontend (http-server)
Git	Cualquiera	Clonar el repositorio y subir cambios

```
Verificar instalaciones con: java -version | mvn -version | node -version | git --version
```

3. Ejecucion en local (desarrollo)

3.1 Iniciar el Backend

1	Ir a la carpeta del backend cd world-cup-predictor/BackEnd
2	Compilar y empaquetar el proyecto mvn clean package -DskipTests
3	Ejecutar el servidor mvn spring-boot:run -- o bien con el jar generado: java -jar target/world-cup-predictor-backend-1.0.0.jar

- | | |
|---|--|
| 4 | Verificar que funciona
Abrir en el navegador: <code>http://localhost:8080/api/teams/list</code>
Debe devolver la lista de equipos en JSON |
|---|--|

3.2 Iniciar el Frontend

- | | |
|---|---|
| 1 | Ir a la carpeta del frontend
<code>cd world-cup-predictor/FrontEnd</code> |
| 2 | Instalar dependencias (solo la primera vez)
<code>npm install</code> |
| 3 | Iniciar el servidor de desarrollo
<code>npm start</code>
El frontend queda disponible en: <code>http://localhost:8000</code> |
| 4 | Abrir la aplicacion
Navegar a <code>http://localhost:8000/html/login.html</code> |

IMPORTANTE: el backend debe estar corriendo ANTES de abrir el frontend. Sin el backend el login y las predicciones no funcionarían.

4. Despliegue en produccion

Se recomienda la siguiente combinacion gratuita: **Netlify** para el frontend y **Railway** para el backend con base de datos PostgreSQL. A continuacion se detalla cada uno.

4.1 Frontend en Netlify (GRATUITO)

- | | |
|---|---|
| 1 | Crear cuenta en Netlify
Ir a <code>netlify.com</code> > Sign up > Continuar con GitHub |
| 2 | Subir el frontend
Opcion A (mas facil): arrastrar la carpeta <code>FrontEnd/</code> al panel de Netlify
Opcion B: conectar el repositorio de GitHub en 'New site from Git' |
| 3 | Configurar la URL del backend
En Netlify > Site settings > Environment variables, agregar:
<code>BACKEND_URL = https://tu-backend.railway.app</code>
(o editar directamente <code>js/config.js</code> con la URL definitiva antes de subir) |
| 4 | Resultado
Netlify genera una URL publica como: <code>https://worldcup-predictor.netlify.app</code> |

4.2 Backend en Railway (plan gratuito con limite mensual)

- | | |
|---|---|
| 1 | Crear cuenta en Railway
Ir a <code>railway.app</code> > Login with GitHub |
|---|---|

2	Crear nuevo proyecto Dashboard > New Project > Deploy from GitHub repo Seleccionar el repositorio y la carpeta BackEnd/
3	Agregar base de datos PostgreSQL En el proyecto > New > Database > PostgreSQL Railway conecta automaticamente las variables de entorno
4	Configurar variables de entorno en Railway En el servicio del backend > Variables, agregar: JWT_SECRET = (una cadena larga y aleatoria, minimo 32 caracteres) JWT_EXPIRATION_MS = 3600000 JWT_REFRESH_EXPIRATION_MS = 86400000 SPRING_PROFILES_ACTIVE = prod
5	Obtener la URL publica En el servicio > Settings > Networking > Generate Domain Copiar la URL (ej: https://backend-produccion.railway.app)

4.3 Migracion de H2 a PostgreSQL para produccion

La base de datos H2 que usa el proyecto actualmente es solo para desarrollo local (se borra al reiniciar el servidor). En produccion se necesita PostgreSQL. Los cambios son minimos:

Paso 1 — Agregar dependencia PostgreSQL en BackEnd/pom.xml:

```
<dependency>
<groupId>org.postgresql</groupId>
<artifactId>postgresql</artifactId>
<scope>runtime</scope>
</dependency>
```

Paso 2 — Crear archivo BackEnd/src/main/resources/application-prod.properties con la configuracion de produccion:

```
spring.datasource.url=${DATABASE_URL}
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
jwt.secret=${JWT_SECRET}
jwt.expiration-ms=${JWT_EXPIRATION_MS}
jwt.refresh-expiration-ms=${JWT_REFRESH_EXPIRATION_MS}
```

Railway proporciona automaticamente la variable DATABASE_URL al agregar PostgreSQL al proyecto.

4.4 Actualizar CORS para produccion

Cuando el frontend este en Netlify y el backend en Railway, hay que actualizar SecurityConfig.java para permitir el dominio de Netlify. En la linea 70 de SecurityConfig.java cambiar:

```
// Antes (desarrollo - permite todo):
configuration.setAllowedOriginPatterns(List.of("*"));

// Despues (produccion - solo tu frontend):
configuration.setAllowedOriginPatterns(List.of(
"https://tu-proyecto.netlify.app",
"http://localhost:8000"
));
```

4.5 Apuntar el frontend a la URL del backend en produccion

En FrontEnd/js/config.js la URL del backend se detecta automaticamente por el hostname del navegador. En produccion esto no funciona porque frontend y backend estan en dominios distintos. Modificar la linea 9 de config.js:

```
// Reemplazar el bloque API_BASE_URL por:  
API_BASE_URL: 'https://tu-backend.railway.app/api',
```

5. Tabla de alternativas de hosting

SERVICIO	PARA QUE	PRECIO	LIMITACIONES EN GRATUITO
Netlify	Frontend estatico	GRATIS	100 GB/mes de trafico, suficiente para un proyecto academico
GitHub Pages	Frontend estatico	GRATIS	Solo archivos estaticos, repositorio debe ser publico
Railway	Backend + BD	~5 USD/mes	Plan gratuito tiene 5 USD de credito mensual
Render	Backend	GRATIS	El servidor duerme tras 15 min sin uso, 750h/mes
Supabase	Base de datos	GRATIS	500 MB de almacenamiento, 2 proyectos activos
Oracle Cloud	Todo en uno (VM)	GRATIS	1 CPU, 1 GB RAM permanente, requiere configuracion manual

6. Checklist antes de publicar

AREA	TAREA
Backend	Cambiar jwt.secret a un valor seguro de al menos 32 caracteres aleatorios
Backend	Activar perfil de produccion (SPRING_PROFILES_ACTIVE=prod)
Backend	Agregar dependencia PostgreSQL en pom.xml y crear application-prod.properties
Backend	Actualizar CORS para incluir solo el dominio de Netlify
Frontend	Cambiar API_BASE_URL en config.js a la URL real del backend en Railway
Frontend	Verificar que DEBUG=false (o quitar los debugLog en produccion)
General	Comprobar que /api/auth/login y /api/matches/predict responden correctamente
General	Probar login, prediccion e historial desde la URL publica de Netlify